

Projektdokumentation zur Sommerprüfung 2026

Fachinformatiker für Anwendungsentwicklung

MDWriter

Modul zur Markdown-Dokumentation in Odoo

Name des Prüflings: Timo Giese
Durchführungszeitraum: 08.05.2026 - 26.05.2026
Projektverantwortlicher: Oliver Kölsch
Prüfende Kammer: IHK Rhein-Neckar

Umschulungsträger	Schulische Ausbildung	Praktikumsbetrieb
Deutsche Rentenversicherung Baden-Württemberg	SRH Berufliche Rehabilitation	TrendTec UG
Mozart-Straße 3 68161 Mannheim	Bonhoeffer-Straße 1 69123 Heidelberg	Mannheimer Straße 105a 68535 Edingen- Neckarhausen

INHALT

ABBILDUNGSVERZEICHNIS	III
TABELLENVERZEICHNIS	III
1 EINLEITUNG	1
1.1 PROJEKTUMFELD	1
1.2 PROJEKTZIEL	1
1.3 PROJEKTBEGRÜNDUNG	1
1.4 PROJEKTABGRENZUNG.....	2
2 PROJEKTPLANUNG	2
2.1 PROJEKTPHASEN UND ZEITPLANUNG	2
2.2 RESSOURCENPLANUNG.....	3
2.3 KOSTENPLANUNG / WIRTSCHAFTLICHKEITSANALYSE	4
3 ANALYSEPHASE	6
3.1 IST-ANALYSE	6
3.2 ANFORDERUNGSANALYSE	7
3.3 USE-CASE-ANALYSE.....	7
4 ENTWURFSPHASE	8
4.1 SYSTEMARCHITEKTUR.....	8
4.2 DATENMODELL	8
4.3 SICHERHEITSKONZEPT	9
4.4 UI/UX-KONZEPT	9
5 IMPLEMENTIERUNGSPHASE	10
5.1 MODULSTRUKTUR	10
5.2 BACKEND-IMPLEMENTIERUNG	10
5.3 FRONTEND-IMPLEMENTIERUNG.....	11
5.4 VERSIONIERUNG UND PDF-EXPORT	11
5.5 SICHERHEITSUMSETZUNG	12
6 TESTPHASE UND ABNAHME	12
6.1 TESTKONZEPT.....	12
6.2 TESTFÄLLE.....	12
6.3 TESTERGEBNISSE UND ABNAHME	13
7 FAZIT	14
7.1 SOLL-IST-VERGLEICH.....	14
7.2 BEWERTUNG DES PROJEKTERGEBNISSES	15
7.3 AUSBLICK	15
8 ANHANG.....	16
A ABKÜRZUNGSVERZEICHNIS	16
B QUELLENVERZEICHNIS	17
C GANTT-DIAGRAM.....	18
D USE-CASE-DIAGRAMM	19
E ARCHITEKTURDIAGRAMM	20
F DATENMODELL.....	21
G QUELLCODEAUSZÜGE	22

G.1 DATENMODELL	22
G.2 AUTOMATISCHE VERSIONIERUNG	23
G.3 OWL-KOMPONENTE / EDITOR-INITIALISIERUNG	23
G.4 ZUGRIFFSSCHUTZ ÜBER ACLS UND RECORD RULES	24
H KUNDENDOKUMENTATION / BEDIENUNGSANLEITUNG	25

ABBILDUNGSVERZEICHNIS

Abbildung 1: TrendTec UG Logo.....	1
Abbildung 2: Anhang C - Gantt-Diagramm der Projektphasen	18
Abbildung 3: Anhang D - Use-Case-Diagramm	19
Abbildung 4: Anhang E - Architekturdiagramm des MDWriter-Moduls	20
Abbildung 5: Anhang F - Datenmodell des MDWriter-Moduls	21

TABELLENVERZEICHNIS

Tabelle 1: Projektphasen und geplante Zeitaufwände	3
Tabelle 2: Personelle Ressourcen	3
Tabelle 3: Eingesetzte Software	4
Tabelle 4: Eingesetzte Hardware	4
Tabelle 5: Kalkulatorische Kosten	4
Tabelle 6: Nutzwertanalyse.....	5
Tabelle 7: Gewichtete Bewertung	5
Tabelle 8: Übersicht des bisherigen Dokumentationsprozesses	6
Tabelle 9: Funktionale Anforderungen	7
Tabelle 10: nicht-funktionale Anforderungen	7
Tabelle 11: Automatisierte Tests	13
Tabelle 12: Manuelle Tests	13
Tabelle 13: Soll / Ist Vergleich	14

1 EINLEITUNG

Die vorliegende Projektdokumentation beschreibt die Konzeption, Entwicklung und Implementierung des Odoo-Moduls MDWriter, das im Rahmen der Umschulung zum Fachinformatiker für Anwendungsentwicklung von Timo Giese bei der TrendTec UG durchgeführt wurde.

1.1 PROJEKTUMFELD

Die TrendTec UG ist ein IT-Dienstleister mit Sitz in Edingen-Neckarhausen und spezialisiert sich auf die Entwicklung sowie Anpassung individueller Softwarelösungen auf Basis des Enterprise-Resource-Planning-Systems Odoo. Das Unternehmen betreut hauptsächlich kleine und mittelständische Kunden bei der Digitalisierung ihrer Geschäftsprozesse und entwickelt dafür maßgeschneiderte Webanwendungen sowie Systemerweiterungen. Aktuell sind 6 Mitarbeiter beschäftigt.

Im Rahmen der täglichen Entwicklungsarbeit entstehen regelmäßig technische Dokumentationen, beispielsweise für Installationsprozesse, API-Beschreibungen, Modulkonfigurationen oder interne Entwicklungsabläufe.

1.2 PROJEKTZIEL

Ziel des Projekts war die Konzeption und Entwicklung eines eigenständigen Odoo-19-Moduls mit dem Namen MDWriter. Das Modul soll technische Dokumentationen direkt in Odoo auf Markdown-Basis erstellen, bearbeiten, versionieren und exportieren können.

Im Fokus standen ein Split-View-Editor mit Live-Vorschau, eine automatische Versionsverwaltung, ein PDF-Export, ein Versionsvergleich sowie eine rollenbasierte Zugriffskontrolle über das bestehende Odoo-Rechtesystem.

1.3 PROJEKTBEGRÜNDUNG

Die bisherige Dokumentationspraxis führte zu Medienbrüchen, uneinheitlichen Ablagestrukturen und erschwerte Nachvollziehbarkeit von Änderungen. Technische Inhalte wurden teilweise in Odoo, teilweise in Git-Wikis oder lokalen Markdown-Dateien gepflegt.

Da technische Dokumentationen im Arbeitsalltag direkt im bestehenden Odoo-System verfügbar sein sollen, wurde ein eigenes Odoo-Modul als passende Lösung gewählt.

1.4 PROJEKTABGRENZUNG

Nicht Bestandteil des Projekts waren Funktionen wie Echtzeit-Zusammenarbeit, externe Cloud-Synchronisation oder ein vollständiges Wiki-System. Ebenfalls nicht vorgesehen waren mobile Anwendungen oder eine eigene Benutzerverwaltung. Die Zugriffskontrolle erfolgt vollständig über das bestehende Odoo-Rechtesystem.

Der Schwerpunkt lag bewusst auf den genannten Kernfunktionen und deren Integration in die bestehende Odoo-Umgebung

2 PROJEKTPLANUNG

Für die Umsetzung wurde das Projekt in die Phasen Analyse, Entwurf, Implementierung, Test und Dokumentation unterteilt.

Es wurde das Wasserfallmodell als Vorgehensmodell gewählt. Die einzelnen Projektphasen bauen dabei aufeinander auf und besitzen klar definierte Ergebnisse und Aufgaben. Aufgrund des festgelegten Projektumfangs, der klaren Zieldefinition sowie des begrenzten zeitlichen Rahmens eignete sich dieses Vorgehensmodell besonders für die Umsetzung des Projekts.

Agile Vorgehensmodelle wie Scrum wurden nicht eingesetzt, da das Projekt allein umgesetzt wurde und keine regelmäßigen Team- oder Sprintabstimmungen erforderlich waren.

2.1 PROJEKTPHASEN UND ZEITPLANUNG

Für die zeitliche Planung wurde das Projekt in mehrere Phasen unterteilt. Die Einteilung orientiert sich am gewählten Vorgehensmodell und berücksichtigt die von der IHK vorgegebene maximale Bearbeitungszeit von 80 Stunden für Anwendungsentwickler. Die geplante Stundenverteilung der einzelnen Projektphasen ist in Tabelle 1 dargestellt.

Die Implementierungsphase nimmt mit 40 Stunden den größten Anteil ein, da hier sowohl die Backend-Logik als auch die Benutzeroberfläche des Moduls umgesetzt werden. Für Analyse und Entwurf wurden zusammen 18 Stunden eingeplant, um die technischen Anforderungen vor der eigentlichen Entwicklung ausreichend zu klären. Zur ergänzenden Darstellung des zeitlichen Ablaufs wurde zusätzlich ein Gantt-Diagramm erstellt. Dieses befindet sich in Anhang C und zeigt die Verteilung der Projektphasen über den gesamten Durchführungszeitraum.

Phase	Aufgabe	Geplante Zeit
Analyse	Ist-Analyse, Zieldefinition, Anforderungsanalyse, Technologierecherche	8 h
Entwurf	Architekturplanung, Datenmodell, Sicherheitskonzept, UI/UX-Konzept	10 h
Implementierung	Backend-Entwicklung, Frontend-Entwicklung, PDF-Export, Versionierung	40 h
Test	Funktionstests, Zugriffstests, Integrationstests, Fehlerbehebung	5 h
Dokumentation	Erstellung der Projektdokumentation und Bedienungsanleitung	17 h
Gesamt		80 h

Tabelle 1: Projektphasen und geplante Zeitaufwände

2.2 RESSOURCENPLANUNG

Für die Umsetzung des Projekts wurden personelle Ressourcen sowie die benötigte Software und Hardware geplant. Da das Projekt im Rahmen der betrieblichen Projektarbeit eigenständig umgesetzt wurde, lag der Hauptaufwand bei mir als Prüfling. Der Projektverantwortliche stand für fachliche Abstimmungen, Freigaben und die spätere Abnahme zur Verfügung.

Personelle Ressource	Aufgabe	Geplanter Aufwand
Timo Giese	Planung, Entwicklung, Test und Dokumentation	80 h
Oliver Kölsch	Fachliche Betreuung, Rückfragen, Abnahme	

Tabelle 2: Personelle Ressourcen

Software	Verwendungszweck
Odoo 19	Zielsystem und Entwicklungsplattform
Visual Studio Code	Quellcodebearbeitung
Git / GitHub	Versionsverwaltung
PostgreSQL	Datenbankmanagementsystem von Odoo
Microsoft Word	Erstellung der Projektdokumentation
Odoo.sh / Entwicklungsserver	Test- und Bereitstellungsumgebung

Tabelle 3: Eingesetzte Software

Hardware	Verwendungszweck
Laptop	Entwicklung, Tests und Dokumentation

Tabelle 4: Eingesetzte Hardware

2.3 KOSTENPLANUNG / WIRTSCHAFTLICHKEITSANALYSE

Da das Projekt im Rahmen meiner Umschulung und des betrieblichen Praktikums durchgeführt wurde, entstanden der TrendTec UG für meine Arbeitszeit keine direkten Personalkosten. Für die Wirtschaftlichkeitsbetrachtung wird der Aufwand dennoch kalkulatorisch bewertet, um die Eigenentwicklung mit möglichen Alternativen vergleichen zu können.

Eine separate Berechnung der Projektbetreuung erfolgt nicht, da fachliche Rückfragen im üblichen Rahmen der betrieblichen Ausbildung stattfanden.

Kostenposition	Berechnung	Betrag
Entwicklungsaufwand	80 h × 24,00 €	1.920,00 €
Gemeinkostenpauschale	10% des Entwicklungsaufwand	192,00 €
Hardware	Vorhandene Arbeitsmittel	0,00 €
Software / Entwicklungsumgebung	Vorhandene bzw. Testzeitraum	0,00 €
Gesamt		2.112,00 €

Tabelle 5: Kalkulatorische Kosten

Da vorhandene Arbeitsmittel genutzt wurden, entstanden keine zusätzlichen Hardware- oder Softwarekosten. Für die Entwicklungsumgebung wurde Odoo.sh über einen zeitlich

begrenzten Testzugang verwendet. Allgemeine Betriebskosten wie Strom, Internet und Arbeitsplatznutzung werden über die Gemeinkostenpauschale berücksichtigt.

Neben der Kostenbetrachtung wurden mögliche Alternativen bewertet. Entscheidend war, ob technische Markdown-Dokumentationen direkt in Odoo erstellt, versioniert und über das bestehende Odoo-Rechtesystem abgesichert werden können. Die bisherige Lösung aus Odoo Knowledge, lokalen Markdown-Dateien und Git-Wikis erfüllt diese Anforderungen nur teilweise. Confluence wurde als externe Dokumentationsplattform betrachtet, ist jedoch nicht nativ in Odoo integrierbar und würde weiterhin Medienbrüche verursachen.

Kriterium	Gewichtung	Bisherige Lösung	Git-Wiki	Confluence	MDWriter
Markdown-Unterstützung	25 %	3	5	2	5
Integration in bestehende Odoo-Umgebung	30 %	2	1	1	5
Versionierung / Nachvollziehbarkeit	20 %	2	4	4	5
Bedienbarkeit im Arbeitsalltag	15 %	2	3	5	4
Kosten / laufender Aufwand	10 %	5	4	2	4
<i>1 = sehr schlecht 2 = schlecht 3 = ausreichend 4 = gut 5 = sehr gut</i>					

Tabelle 6: Nutzwertanalyse

Lösung	Berechnung	Ergebnis
Bisherige Lösung	$(3 \times 25) + (2 \times 30) + (2 \times 20) + (2 \times 15) + (5 \times 10)$	255 / 500
Git-Wiki	$(5 \times 25) + (1 \times 30) + (4 \times 20) + (3 \times 15) + (4 \times 10)$	320 / 500
Confluence	$(2 \times 25) + (1 \times 30) + (5 \times 20) + (4 \times 15) + (2 \times 10)$	255 / 500
MDWriter	$(5 \times 25) + (5 \times 30) + (5 \times 20) + (4 \times 15) + (4 \times 10)$	475 / 500

Tabelle 7: Gewichtete Bewertung

Die Nutzwertanalyse zeigt, dass MDWriter mit 475 von 500 Punkten den höchsten Nutzwert erreicht. Ausschlaggebend ist die direkte Integration in Odoo, wodurch Dokumente zentral verwaltet, versioniert und über das bestehende Berechtigungssystem abgesichert werden können.

Confluence eignet sich grundsätzlich als Dokumentationsplattform, würde jedoch ein zusätzliches externes System neben Odoo einführen. Dadurch blieben die bestehenden Medienbrüche zwischen Odoo und der Dokumentation weiterhin bestehen.. Zusätzlich fallen bei größeren Teams laufende Lizenzkosten an; der Standard-Tarif wird auf der offiziellen Atlassian-Preisseite mit 5,42 US-Dollar pro Benutzer und Monat angegeben (Atlassian, 2026).

Aus der Kostenbetrachtung und der Nutzwertanalyse ergab sich die Eigenentwicklung von MDWriter als sinnvollste Lösung.

3 ANALYSEPHASE

In der Analysephase wurde zunächst die bestehende Dokumentationssituation innerhalb der TrendTec UG betrachtet. Ziel war es, die aktuellen Schwachstellen zu erkennen und daraus konkrete Anforderungen an die neue Lösung abzuleiten.

3.1 IST-ANALYSE

Dabei zeigte sich, dass technische Dokumentationen bisher auf mehrere Werkzeuge verteilt waren und nicht zentral in Odoo gepflegt wurden.

Die folgende Tabelle fasst die verwendeten Werkzeuge und deren Einschränkungen zusammen:

Werkzeug	Verwendung	Einschränkung
Odoo Knowledge / Odoo-Editor	kurze Notizen und einfache Inhalte	keine geeignete Markdown-Bearbeitung für technische Dokumentation
Git-Wiki / Git-Repositories	technische Markdown-Dokumentation	nicht direkt in Odoo integriert
Lokale Markdown-Dateien	persönliche oder projektbezogene Notizen	keine zentrale Ablage und keine einheitliche Versionierung
PDF-Dateien	Weitergabe fertiger Dokumentationen	nachträgliche Bearbeitung nur eingeschränkt möglich

Tabelle 8: Übersicht des bisherigen Dokumentationsprozesses

Aus der Ist-Analyse ergab sich der Bedarf an einer zentralen Lösung für technische Markdown-Dokumentationen innerhalb von Odoo. Wesentliche Anforderungen waren

dabei eine direkte Bearbeitung mit Live-Vorschau, eine nachvollziehbare Versionierung, Zugriffsschutz und Exportmöglichkeiten.

3.2 ANFORDERUNGSANALYSE

Nr.	Anforderung	Priorität
F01	Markdown-Editor mit Live-Vorschau	Muss
F02	Speicherung der Dokumente in Odoo	Muss
F03	automatische Versionierung	Muss
F04	Zugriffsschutz über Odoo-Rechte	Muss
F05	PDF-Export	Muss
F06	Versionsvergleich	Soll
F07	Statusverwaltung	Soll

Tabelle 9: Funktionale Anforderungen

Nr.	Anforderung
NF01	Kompatibilität mit Odoo 19
NF02	einfache Bedienbarkeit
NF03	keine externen Laufzeitdienste
NF04	sichere Verarbeitung von Markdown-Inhalten
NF05	wartbare Modulstruktur

Tabelle 10: nicht-funktionale Anforderungen

3.3 USE-CASE-ANALYSE

Ein Benutzer kann eigene Dokumente erstellen, bearbeiten, versionieren, vergleichen und exportieren. Ein Administrator besitzt darüber hinaus erweiterte Rechte und kann alle Dokumente einsehen, verwalten und löschen.

Im Use-Case-Diagramm wird der Administrator als spezialisierter Benutzer dargestellt. Dies beschreibt keine technische Vererbung im Quellcode, sondern die fachliche Rollenstruktur innerhalb des Systems.

Das vollständige Use-Case-Diagramm befindet sich in Anlage D.

4 ENTWURFSPHASE

In der Entwurfsphase wurden die technischen Grundlagen für die spätere Umsetzung des Moduls festgelegt. Dazu gehörten die Systemarchitektur, das Datenmodell, das Sicherheitskonzept sowie die Gestaltung der Benutzeroberfläche.

Da MDWriter als integriertes Odoo-19-Modul umgesetzt wird, orientiert sich der Entwurf an den bestehenden Odoo-Konventionen. Dadurch können vorhandene Mechanismen wie ORM, Benutzerrechte, Attachments und Reporting genutzt werden.

4.1 SYSTEMARCHITEKTUR

MDWriter wurde als integriertes Odoo-19-Modul entworfen und in die bestehende Odoo-Architektur eingebunden. Die Speicherung der Daten erfolgt über PostgreSQL als Datenbankmanagementsystem und über das Odoo-ORM, das den Zugriff auf die Datenbank über Python-Modelle ermöglicht.

Die Modullogik wird in Python umgesetzt. Dazu gehören unter anderem das Speichern, Versionieren und Exportieren der Dokumente. Die Benutzeroberfläche wird als OWL-Komponente im Odoo-Webclient bereitgestellt und bindet den Markdown-Editor direkt in die Odoo-Oberfläche ein. (Odoo S.A., 2026)

Für die Markdown-Bearbeitung wird CodeMirror (CodeMirror, 2026) als Editor genutzt. Die Live-Vorschau wird mit markdown-it (markdown-it, 2026) direkt im Browser erzeugt. Für den PDF-Export wird Mistune (Mistune, 2026) serverseitig eingesetzt, um den Markdown-Inhalt vor der PDF-Erzeugung in HTML umzuwandeln.

Das Architekturdiagramm des Moduls ist in Anhang E dargestellt.

4.2 DATENMODELL

Das Datenmodell besteht aus dauerhaft gespeicherten Odoo-Modellen und einem Hilfsmodell für den Versionsvergleich. Das Hauptmodell `x.md.document` bildet ein Markdown-Dokument ab und speichert unter anderem Titel, Inhalt, Status, Eigentümer und die aktuelle Versionsnummer. Das Präfix `x.` kennzeichnet in Odoo eigene Module, die nicht zum Odoo-Kern gehören.

Das Versionsmodell `x.md.document.version` speichert einzelne Dokumentversionen mit

Inhalt, Versionsnummer, Änderungszeitpunkt, Prüfsumme und optionalen Datei-Anhängen. Ein Auszug der wichtigsten Felder befindet sich in Anhang G.1.

Die Trennung zwischen Dokument und Version ermöglicht eine nachvollziehbare Historie. Änderungen am Dokument erzeugen neue Versionsdatensätze, während bestehende Versionen nicht nachträglich verändert werden. Für den Versionsvergleich wird ein Hilfsmodell verwendet, das keine eigenen Daten dauerhaft speichert, sondern nur während des Vergleichs aktiv ist. In Odoo werden solche temporären Modelle über „TransientModel“ umgesetzt. (Odoo S.A., 2026)

Das Datenmodell ist in Anhang F dargestellt.

4.3 SICHERHEITSKONZEPT

Das Sicherheitskonzept nutzt die vorhandenen Berechtigungsmechanismen von Odoo, um den Zugriff auf Dokumente und Versionen zu steuern. Über Zugriffskontrolllisten (ACLs) wird festgelegt, welche Aktionen ein Benutzer auf einem Modell ausführen darf. Normale Benutzer dürfen eigene Dokumente erstellen, lesen und bearbeiten, jedoch nicht löschen. Versionseinträge sind nur lesbar, damit die Versionshistorie nicht nachträglich verändert werden kann. Administratoren dürfen zusätzlich Dokumente löschen und haben Zugriff auf alle Dokumente im System.

Da ACLs nur allgemein für ein ganzes Modell gelten, werden zusätzlich Record Rules eingesetzt. Diese legen fest, welche einzelnen Datensätze ein Benutzer sehen darf. Normale Benutzer sehen ausschließlich Dokumente, deren Eigentümer sie selbst sind. Administratoren erhalten uneingeschränkten Zugriff auf alle Dokumente.

Damit kein schädlicher Code über Markdown-Inhalte eingeschleust werden kann, wird eingebettetes HTML nicht direkt ausgeführt, sondern durch die eingesetzten Markdown-Renderer verarbeitet. (OWASP Foundation, 2026)

4.4 UI/UX-KONZEPT

Für die Benutzeroberfläche wurde ein Split-View-Konzept gewählt. Links befindet sich der Markdown-Editor mit Syntaxunterstützung, rechts wird eine Live-Vorschau des gerenderten Dokuments angezeigt. Dadurch können Änderungen direkt während der Bearbeitung überprüft werden.

Die Oberfläche wurde an das bestehende Odoo-Design angepasst und um ein dezentes TrendTec-Branding ergänzt. Zusätzlich wurde auf eine gute Lesbarkeit technischer Inhalte geachtet, beispielsweise durch geeignete Schriftarten für Code und Fließtext. Das Modul unterstützt außerdem den Dark Mode von Odoo und stellt den Dokumentstatus in Listen- und Formularansichten visuell dar.

Der Diff-Wizard ermöglicht den visuellen Vergleich zweier Versionen direkt im Browser, wobei Änderungen farblich hervorgehoben werden.

5 IMPLEMENTIERUNGSPHASE

In der Implementierungsphase wurde das Modul MDWriter auf Grundlage des zuvor erstellten Entwurfs umgesetzt. Dabei wurden Backend-Modelle, Sicherheitsregeln, Views, eine eigene Frontend-Komponente sowie Funktionen für Versionierung und PDF-Export implementiert.

5.1 MODULSTRUKTUR

Das Modul trägt den technischen Namen `markdown_editor` und folgt der Standardstruktur eines Odoo-Moduls. Die wichtigsten Bestandteile sind die Python-Modelle für Dokumente und Versionen, XML-Views für Listen- und Formularansichten, Sicherheitsdateien für ACLs und Record Rules sowie statische Frontend-Dateien für den Markdown-Editor.

Eine visuelle Darstellung der Verzeichnisstruktur ist in Anhang G zu sehen.

Die Frontend-Bibliotheken `CodeMirror` und `markdown-it` wurden lokal im Modul eingebunden. Dadurch ist das Modul nicht von externen Content-Delivery-Netzwerken (CDNs) abhängig und kann auch in abgeschotteten oder intern betriebenen Odoo-Umgebungen genutzt werden. Zusätzlich enthält das Modul eine Vorlage für den PDF-Export sowie einen Assistenten für den Versionsvergleich.

5.2 BACKEND-IMPLEMENTIERUNG

Die Backend-Logik wurde in Python mit dem Odoo-ORM (Odoo S.A., 2026) umgesetzt. Das Hauptmodell `x.md.document` speichert den aktuellen Dokumentstand mit Titel, Markdown-Inhalt, Status und Eigentümer. Beim Anlegen eines Dokuments wird der aktuell angemeldete Benutzer automatisch als Eigentümer gesetzt.

Für die Versionshistorie wurde das Modell `x.md.document.version` implementiert. Bei der Erstellung eines Dokuments sowie bei Änderungen am Markdown-Inhalt wird automatisch ein neuer Versionsdatensatz erzeugt. Dadurch bleiben frühere Bearbeitungsstände nachvollziehbar erhalten. Zusätzlich wird für jede Version eine

Prüfsumme gebildet, um Änderungen eindeutig identifizieren zu können.

Die Versionierung wurde so umgesetzt, dass bestehende Versionen nicht nachträglich verändert werden.

Die zentrale Implementierung der automatischen Versionierung ist als Quellcodeauszug in Anhang G.2 dargestellt.

5.3 FRONTEND-IMPLEMENTIERUNG

Für die Bearbeitung der Markdown-Dokumente wurde eine eigene OWL-Komponente entwickelt. Dabei handelt es sich um einen Frontend-Baustein innerhalb des Odoo-Webclients. Diese Komponente bindet den CodeMirror-Editor in die Formularansicht ein und stellt den Split-View bereit: links befindet sich die Markdown-Eingabe, rechts wird eine Live-Vorschau des gerenderten Inhalts angezeigt.

Die Live-Vorschau wird im Browser mit markdown-it erzeugt. Zur Verbesserung der Bedienbarkeit wird die Vorschau nicht bei jedem einzelnen Tastendruck sofort neu aufgebaut, sondern zeitverzögert aktualisiert. Dadurch bleibt die Oberfläche auch bei längeren Dokumenten flüssig bedienbar.

Das Styling wurde auf CSS-Klassen des eigenen Moduls beschränkt. Dadurch wird verhindert, dass die Darstellung anderer Odoo-Module unbeabsichtigt verändert wird. Zusätzlich wurde die Darstellung an das Odoo-Layout angepasst und für die Nutzung im Dark Mode vorbereitet.

Ein Auszug der OWL-Komponente zur Initialisierung des Editors befindet sich in Anhang G.3. Die Initialisierung ist erforderlich, weil CodeMirror ein bereits geladenes Eingabefeld benötigt. Deshalb wird der Editor erst gestartet, nachdem die OWL-Komponente in der Odoo-Oberfläche eingebunden wurde.

5.4 VERSIONIERUNG UND PDF-EXPORT

Für den PDF-Export wird der Markdown-Inhalt serverseitig in HTML umgewandelt und anschließend über den Odoo-Reportingmechanismus als PDF ausgegeben. Bereits erzeugte PDF-Dateien werden als Anhang gespeichert und bei erneutem Export wiederverwendet.

Falls beim PDF-Rendering ein Fehler auftritt, wird dieser protokolliert, ohne die Versionierung zu unterbrechen. Dadurch bleibt die Dokumenthistorie auch dann erhalten, wenn die PDF-Erzeugung vorübergehend fehlschlägt.

5.5 SICHERHEITSUMSETZUNG

Die in der Entwurfsphase geplante Zugriffskontrolle wurde über Odoo-ACLs und Record Rules umgesetzt. ACLs regeln die grundsätzlichen Rechte auf die Modelle, während Record Rules den Zugriff auf einzelne Dokumente einschränken.

Normale Benutzer können dadurch nur eigene Dokumente sehen und bearbeiten. Das Löschen von Dokumenten sowie der Zugriff auf alle Dokumente bleibt Administratoren vorbehalten. Die Versionshistorie ist für normale Benutzer nur lesbar, damit frühere Versionen nicht manuell verändert werden können.

Auszüge der ACLs und Record Rules befinden sich in Anhang G.4.

6 TESTPHASE UND ABNAHME

6.1 TESTKONZEPT

Zur Qualitätssicherung wurden automatisierte Backend-Tests sowie manuelle Integrationstests in der Odoo-Entwicklungsumgebung durchgeführt. Die automatisierten Tests prüften insbesondere die Versionierung, die Zugriffskontrolle und den Versionsvergleich. Ergänzend wurden die wichtigsten Benutzerfunktionen direkt im Browser getestet, um das Zusammenspiel von Backend, Benutzeroberfläche und Odoo-Rechtesystem zu überprüfen.

6.2 TESTFÄLLE

Die automatisierten Tests wurden mit dem Odoo-Testframework umgesetzt. Dabei handelt es sich um eine von Odoo bereitgestellte Testumgebung, mit der Modelle, Berechtigungen und Funktionen automatisiert geprüft werden können. (Odoo S.A., 2026)

Nr.	Testfall	Erwartetes Ergebnis
T1	Neues Dokument anlegen	Version 1 wird automatisch erstellt
T2	Dokumentinhalt ändern	neue Version wird erzeugt
T3	ältere Version wiederherstellen	ursprünglicher Inhalt wird wiederhergestellt

T4	eigenes Dokument öffnen	Zugriff ist erlaubt
T5	fremdes Dokument öffnen	Zugriff wird durch Record Rule verhindert
T6	Version manuell anlegen	Zugriff wird verweigert
T7	zwei Versionen vergleichen	Unterschiede werden im Diff angezeigt

Tabelle 11: Automatisierte Tests

Zusätzlich wurden folgende manuelle Tests durchgeführt:

Testbereich	Prüfung
Dokumentbearbeitung	Anlegen, Bearbeiten und Speichern eines Dokuments
Live-Vorschau	Darstellung von Überschriften, Listen, Codeblöcken, Tabellen und Links
PDF-Export	korrekte Ausgabe des Dokuments als PDF
Versionsvergleich / Diff-Wizard	farbliche Hervorhebung von Änderungen
Zugriffsschutz	Anmeldung mit separatem Testbenutzer
Darstellung	Prüfung der Oberfläche im normalen Modus und im Dark Mode

Tabelle 12: Manuelle Tests

6.3 TESTERGEBNISSE UND ABNAHME

Alle automatisierten Testfälle wurden erfolgreich ausgeführt. Auch die manuellen Integrationstests bestätigten die grundlegende Funktion des Moduls.

Während der Tests traten kleinere Anpassungen auf. Beispielsweise waren CSS-Regeln anfangs zu allgemein definiert, wodurch auch andere Odoo-Ansichten beeinflusst wurden. Dies wurde korrigiert, indem das Styling auf eigene Modulklassen beschränkt wurde. Außerdem wurde die Aktualisierung der Live-Vorschau zeitverzögert umgesetzt, damit die Oberfläche bei längeren Dokumenten flüssig bedienbar bleibt.

Im Rahmen der Tests konnte bestätigt werden, dass die zentralen Anforderungen des Moduls erfüllt wurden. Dazu zählen insbesondere die automatische Versionierung, die

Zugriffsbeschränkung auf eigene Dokumente, der PDF-Export sowie der visuelle Vergleich unterschiedlicher Dokumentversionen.

Die Abnahme erfolgte durch den Projektverantwortlichen Oliver Kölsch. Dabei wurden die Kernfunktionen anhand der definierten Anforderungen überprüft. Nach erfolgreicher Prüfung wurde das Modul für den internen Einsatz freigegeben.

7 FAZIT

7.1 SOLL-IST-VERGLEICH

Zum Abschluss des Projekts wurden die geplanten Anforderungen mit dem tatsächlich erreichten Projektergebnis verglichen.

Anforderung	Soll	Ist	Ergebnis
Markdown-Editor	Dokumente sollen direkt in Odoo mit Markdown bearbeitet werden können	umgesetzt mit CodeMirror	erfüllt
Live-Vorschau	Änderungen sollen während der Bearbeitung sichtbar sein	umgesetzt mit markdown-it im Browser	erfüllt
Speicherung in Odoo	Dokumente sollen zentral in Odoo gespeichert werden	umgesetzt über eigene Odoo-Modelle	erfüllt
Versionierung	Änderungen sollen automatisch nachvollziehbar gespeichert werden	automatische Versionserstellung bei Änderungen	erfüllt
Versionsvergleich	Unterschiede zwischen Versionen sollen sichtbar sein	umgesetzt über Diff-Wizard	erfüllt
PDF-Export	Dokumente sollen als PDF exportiert werden können	umgesetzt über Odoo-Reporting	erfüllt
Zugriffsschutz	Benutzer sollen nur eigene Dokumente sehen und bearbeiten können	umgesetzt über ACLs und Record Rules	erfüllt
Statusverwaltung	Dokumente sollen unterschiedliche Bearbeitungsstände abbilden können	umgesetzt mit den Status Entwurf, Veröffentlicht und Archiviert	erfüllt

Tabelle 13: Soll / Ist Vergleich

Das Projekt wurde innerhalb des geplanten Zeitrahmens von 80 Stunden abgeschlossen. Kleinere Verschiebungen ergaben sich innerhalb der Implementierungsphase. Die Backend-Entwicklung verlief zügiger als erwartet, während die Einbindung des Markdown-Editors in die Odoo-Oberfläche mehr Zeit beanspruchte.

7.2 BEWERTUNG DES PROJEKTERGEBNISSES

Das Projektziel wurde erreicht. Mit MDWriter steht ein Odoo-Modul zur Verfügung, mit dem technische Dokumentationen direkt in Odoo erstellt, bearbeitet, versioniert, verglichen und als PDF exportiert werden können.

Besonders positiv ist die direkte Integration in die bestehende Odoo-Umgebung. Dadurch können vorhandene Mechanismen wie Benutzerverwaltung, Berechtigungssystem, Datenhaltung und Reporting genutzt werden, ohne ein zusätzliches externes System einzuführen. Die zuvor bestehenden Medienbrüche zwischen Odoo, lokalen Markdown-Dateien und Git-Wikis konnten dadurch reduziert werden.

Während der Umsetzung zeigte sich, dass vor allem die Frontend-Integration anspruchsvoll war. Die Einbindung von CodeMirror in eine OWL-Komponente sowie die Anpassung an das Odoo-Layout erforderten mehrere Tests und Korrekturen. Gleichzeitig konnte ich dadurch mein Verständnis für die Entwicklung von Benutzeroberflächen in Odoo vertiefen.

7.3 AUSBLICK

Das Modul kann als Grundlage für die interne technische Dokumentation in Odoo genutzt werden. Für zukünftige Erweiterungen wären zusätzliche Funktionen denkbar, beispielsweise Vorlagen für Installationsanleitungen, API-Dokumentationen oder Modulbeschreibungen, eine erweiterte Suche im Markdown-Inhalt sowie Kommentarfunktionen zu einzelnen Versionen.

Mittelfristig könnte MDWriter auch in Kundenprojekten eingesetzt werden, um projektbezogene technische Dokumentationen direkt innerhalb von Odoo zu verwalten. Funktionen wie Freigabeprozesse oder eine stärkere Anpassung des Brandings wurden im Rahmen dieses Projekts bewusst nicht umgesetzt, könnten aber bei Bedarf ergänzt werden.

8 ANHANG

A ABKÜRZUNGSVERZEICHNIS

ABKÜRZUNG	BEDEUTUNG
ACL	Access Control List
API	Application Programming Interface
CDN	Content Delivery Network
CSS	Cascading Style Sheets
ERP	Enterprise-Resource-Planning-System
HTML	Hypertext Markup Language
IHK	Industrie- und Handelskammer
MD	Markdown
ORM	Object-Relational Mapping
OWL	Odoo Web Library
PDF	Portable Document Format
SQL	Structured Query Language
UI	User Interface
UX	User Experience
XML	Extensible Markup Language
XSS	Cross-Site Scripting

B QUELLENVERZEICHNIS

- Atlassian. (20. Mai 2026). *Atlassian*. Von Confluence Pricing: <https://www.atlassian.com/software/confluence/pricing> abgerufen
- CodeMirror. (2026). *CodeMirror*. Von CodeMirror - Versatile text editor: <https://codemirror.net/> abgerufen
- markdown-it. (2026). *markdown-it*. Von markdown-it - Markdown parser: <https://github.com/markdown-it/markdown-it>; <https://markdown-it.github.io/> abgerufen
- Mistune. (2026). *Mistune documentation*. Von Mistune - A fast yet powerful Python Markdown parser: <https://mistune.lepture.com/en/latest/> abgerufen
- Odoo S.A. (2026). *Chapter 1: Architecture Overview*. Von Odoo 19.0 Documentation: https://www.odoo.com/documentation/19.0/developer/tutorials/server_framework_101/ abgerufen
- Odoo S.A. (2026). *ORM API*. Von Odoo 19.0 Documentation: <https://www.odoo.com/documentation/19.0/developer/reference/backend/orm.html> abgerufen
- Odoo S.A. (2026). *Testing Odoo*. Von Odoo Documentation: <https://www.odoo.com/documentation/19.0/developer/reference/backend/testing.html> abgerufen
- OWASP Foundation. (2026). *Cross Site Scripting (XSS)*. Von OWASP: <https://owasp.org/www-community/attacks/xss/> abgerufen

C GANTT-DIAGRAM

Projektzeitraum: 08.05.2026 – 26.05.2026

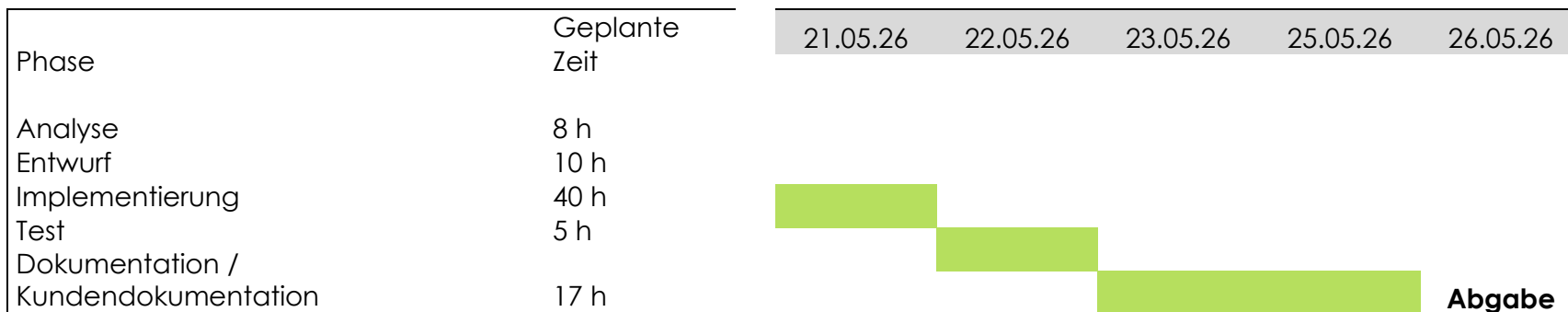
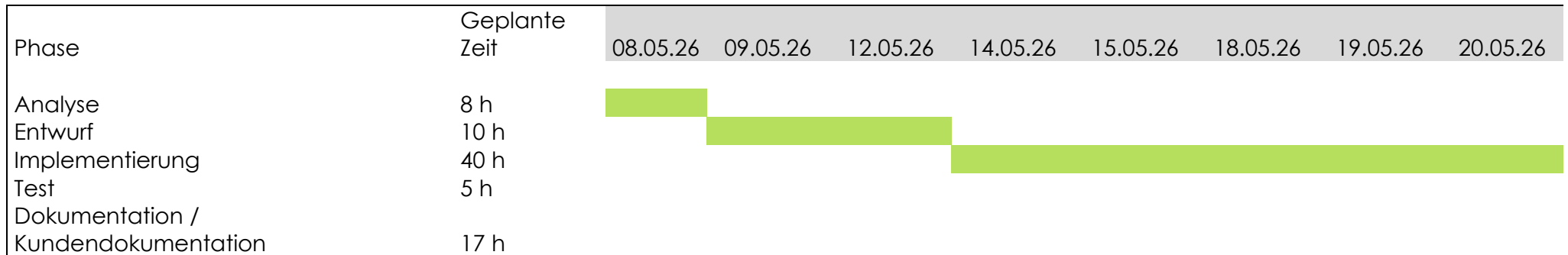


Abbildung 2: Anhang C - Gantt-Diagramm der Projektphasen

D USE-CASE-DIAGRAMM

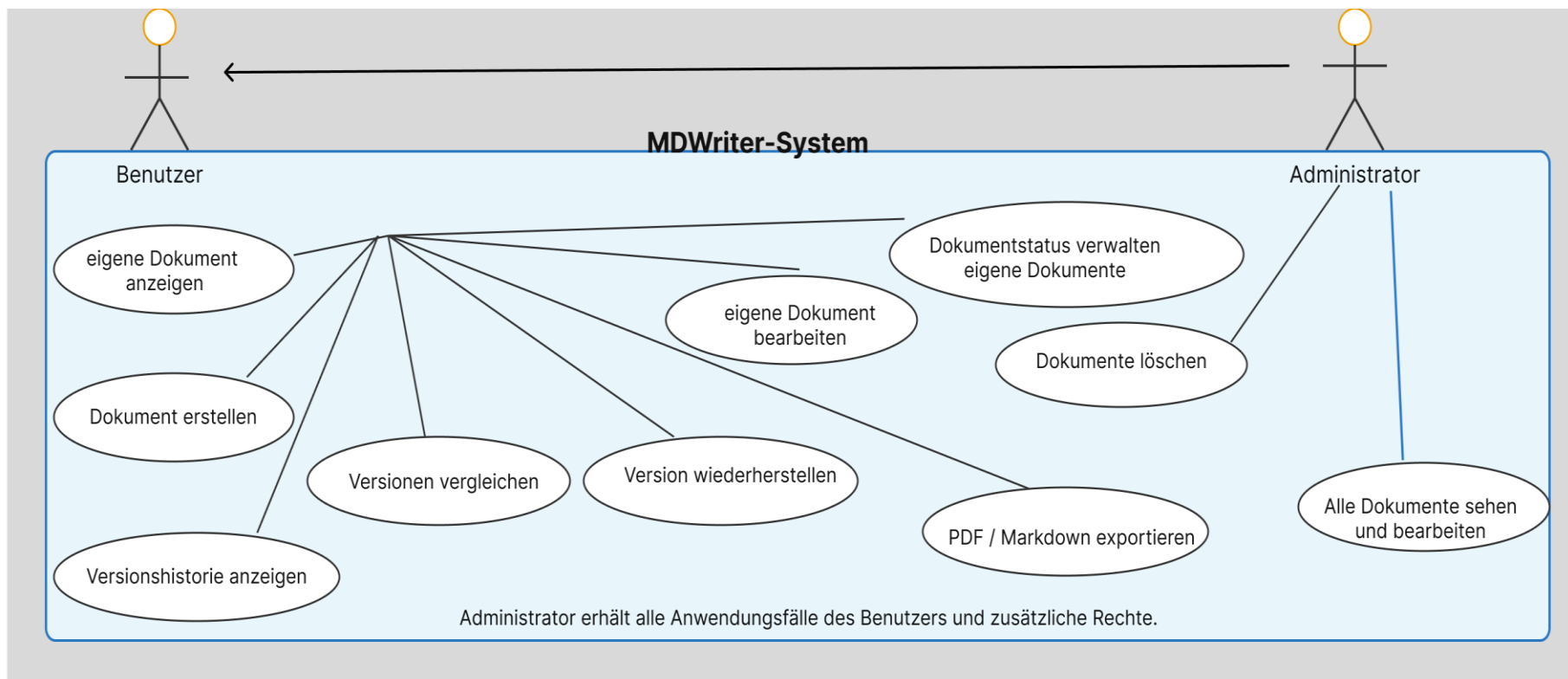


Abbildung 3: Anhang D - Use-Case-Diagramm

E ARCHITEKTURDIAGRAMM

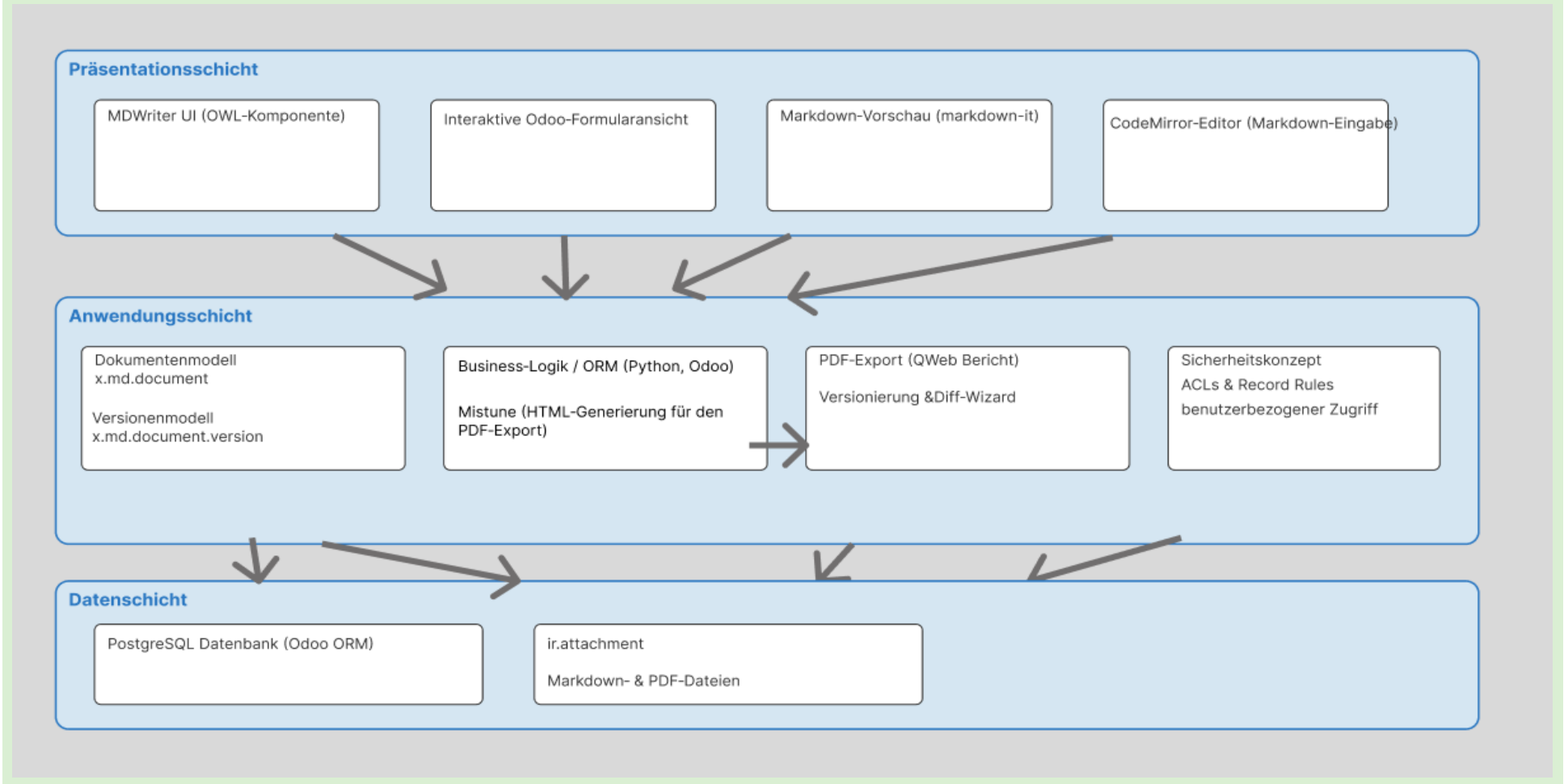


Abbildung 4: Anhang E - Architekturdiagramm des MDWriter-Moduls

F DATENMODELL

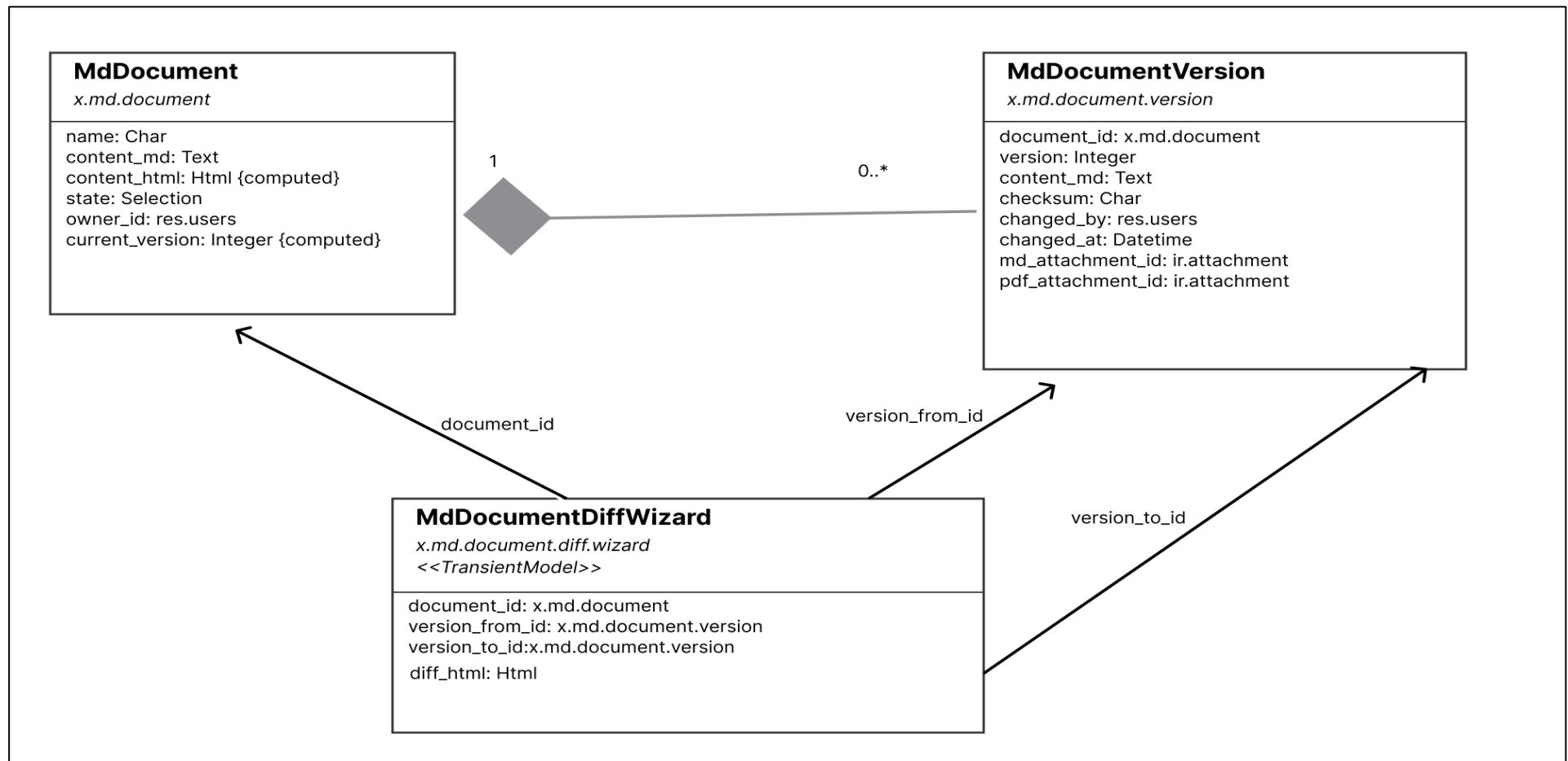


Abbildung 5: Anhang F - Datenmodell des MDWriter-Moduls

G QUELLCODEAUSZÜGE

Der folgende Auszug zeigt die wichtigsten Bestandteile der Modulstruktur.

```
markdown_editor/
├── models/
│   ├── md_document.py           # Dokument- und Versionsmodell
│   └── md_document_diff.py      # Diff-Wizard
├── views/                       # XML-Views (Formular, Liste, Suche)
├── security/                    # ACL und Record Rules
├── report/                      # PDF-Export-Vorlage
├── static/
│   ├── lib/
│   │   ├── codemirror/         # CodeMirror (lokal eingebunden)
│   │   └── markdown-it.min.js  # markdown-it (lokal eingebunden)
│   └── src/
│       ├── js/                 # OWL-Komponente
│       ├── scss/               # Styling
│       └── fonts/              # Lokale Schriftarten
└── __manifest__.py             # Modulmetadaten
```

G.1 DATENMODELL

Datei: markdown_editor/models/md_document.py

Der folgende Ausschnitt zeigt die wichtigsten Felder des Dokumentmodells. Das Modell speichert den Titel, den Markdown-Inhalt, den Status, den Eigentümer sowie die zugehörigen Versionen.

```
class XMdDocument(models.Model):
    _name = "x.md.document"

    name = fields.Char(string="Titel", required=True)
    content_md = fields.Text(string="Markdown-Inhalt")
    state = fields.Selection([
        ("draft", "Entwurf"),
        ("published", "Veröffentlicht"),
        ("archived", "Archiviert"),
    ], default="draft")
    owner_id = fields.Many2one(
        "res.users",
        default=lambda self: self.env.user
    )
    version_ids = fields.One2many(
        "x.md.document.version",
        "document_id"
    )
    current_version = fields.Integer(
        compute="_aktuelle_version_berechnen",
        store=True
    )
)
```

G.2 AUTOMATISCHE VERSIONIERUNG

Datei: markdown_editor/models/md_document.py

Der folgende Ausschnitt zeigt die automatische Versionierung. Beim Speichern eines geänderten Markdown-Inhalts wird eine neue Version des Dokuments angelegt.

```
def write(self, vals):
    res = super().write(vals)

    # Nur bei Änderungen am Markdown-Inhalt wird eine neue Version angelegt.
    if "content_md" in vals:
        self._version_anlegen()

    return res

def _version_anlegen(self):
    Version = self.env["x.md.document.version"].sudo()

    for record in self:
        content = record.content_md or ""

        Version.create({
            "document_id": record.id,
            "version": record.current_version + 1,
            "content_md": content,
            "checksum": hashlib.md5(content.encode("utf-8")).hexdigest(),
            "changed_by": self.env.user.id,
            "changed_at": fields.Datetime.now(),
            "md_attachment_id": self._md_datei_speichern(record, content),
            "pdf_attachment_id": self._pdf_datei_speichern(record) or False,
        })
```

G.3 OWL-KOMPONENTE / EDITOR-INITIALISIERUNG

Datei: markdown_editor/static/src/js/markdown_editor.js

Der Ausschnitt zeigt, warum CodeMirror erst nach dem Laden der Komponente initialisiert wird.

```

onMounted(() => {
  this.cm = window.CodeMirror.fromTextArea(this.editorRef.el, {
    mode: "markdown",
    lineWrapping: true,
    theme: "default",
    readOnly: this.props.readonly ? "nocursor" : false,
  });

  this.cm.setValue(this.state.value);

  this.cm.on("change", (cm) => {
    clearTimeout(this._debounce);
    this._debounce = setTimeout(
      () => this._updateState(cm.getValue()),
      300
    );
  });
});

onWillUnmount(() => {
  clearTimeout(this._debounce);
  if (this.cm) {
    this.cm.toTextArea();
    this.cm = null;
  }
});

```

G.4 ZUGRIFFSSCHUTZ ÜBER ACLS UND RECORD RULES

Datei: markdown_editor/security/ir.model.access.csv

```

id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_md_document_user,md.document.user,model_x_md_document,base.group_user,1,1,1,0
access_md_document_admin,md.document.admin,model_x_md_document,base.group_system,1,1,1,1
access_md_document_version_user,md.version.user,model_x_md_document_version,base.group_user,1,0,0,0

```

id	Modell	Gruppe	Lesen	Schreiben	Erstellen	Löschen
access_md_document_user	x.md.document	group_user	1	1	1	0
access_md_document_admin	x.md.document	group_system	1	1	1	1
access_md_document_version_user	x.md.document.version	group_user	1	0	0	0

Datei: markdown_editor/security/markdown_editor_security.xml

```
<record id="markdown_document_rule_owner_write" model="ir.rule">
  <field name="name">Benutzer sieht nur eigene Dokumente</field>
  <field name="model_id" ref="model_x_md_document"/>
  <field name="domain_force">[("owner_id", "=", user.id)]</field>
  <field name="groups" eval="[(4, ref('base.group_user'))]">/>
</record>

<record id="markdown_document_rule_admin" model="ir.rule">
  <field name="name">Administrator sieht alle Dokumente</field>
  <field name="model_id" ref="model_x_md_document"/>
  <field name="domain_force">[(1, "=", 1)]</field>
  <field name="groups" eval="[(4, ref('base.group_system'))]">/>
</record>
```

H KUNDENDOKUMENTATION / BEDIENUNGSANLEITUNG

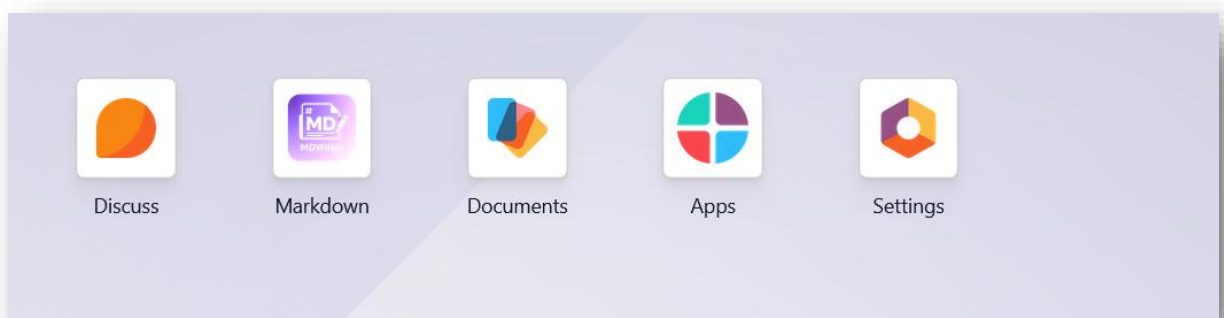
H.1 Ziel der Anwendung

MDWriter dient zur Erstellung und Verwaltung technischer Markdown-Dokumentationen innerhalb von Odoo. Benutzer können eigene Dokumente erstellen, bearbeiten, versionieren, vergleichen und als PDF exportieren.

H.2 Modul aufrufen

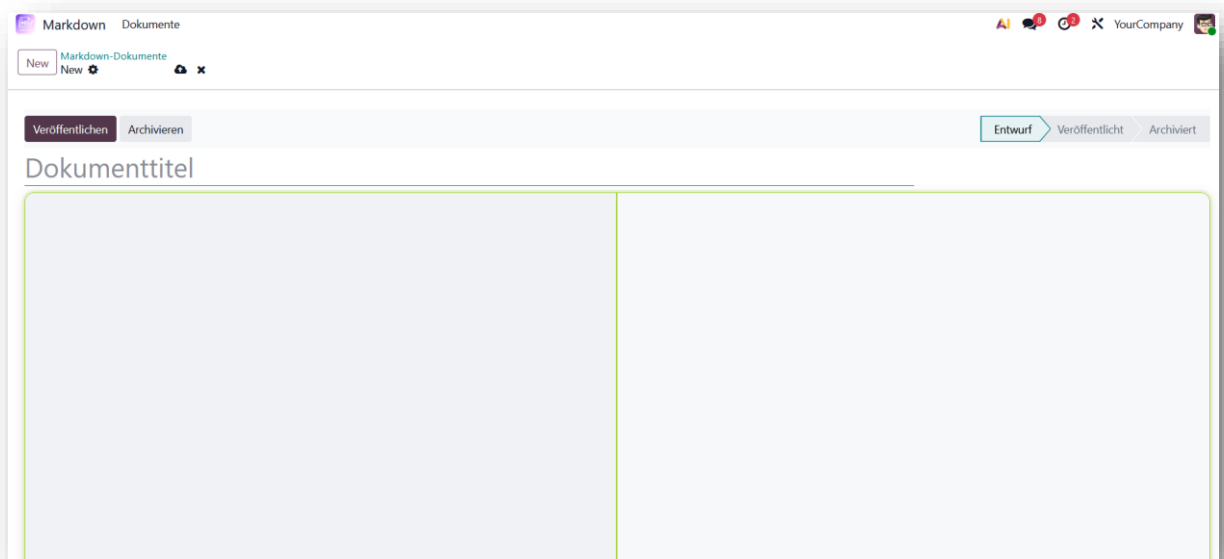
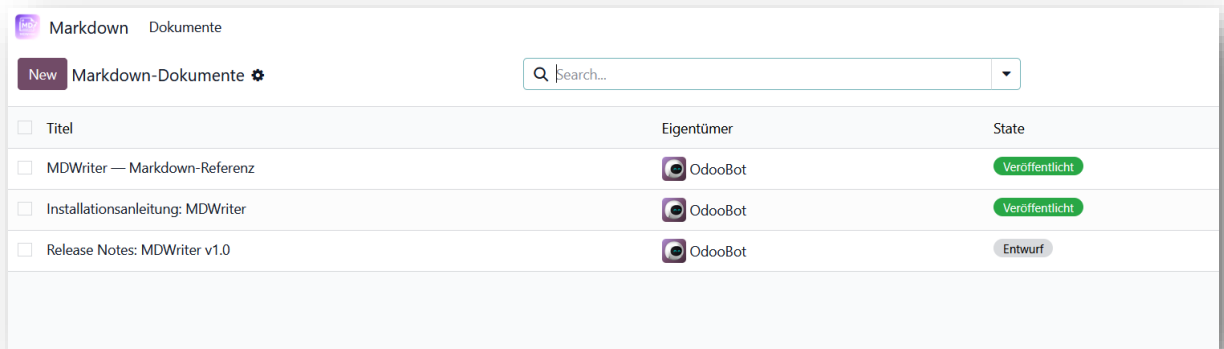
Für die Nutzung muss das Modul MDWriter in einer bestehenden Odoo-19-Instanz installiert sein. Die Benutzer müssen einer passenden Odoo-Benutzergruppe zugeordnet sein.

Nach der Anmeldung in Odoo wird das Modul über das App-Menü geöffnet. Dort erscheint die Übersicht der vorhandenen Markdown-Dokumente. Normale Benutzer sehen nur eigene Dokumente, Administratoren können alle Dokumente einsehen.



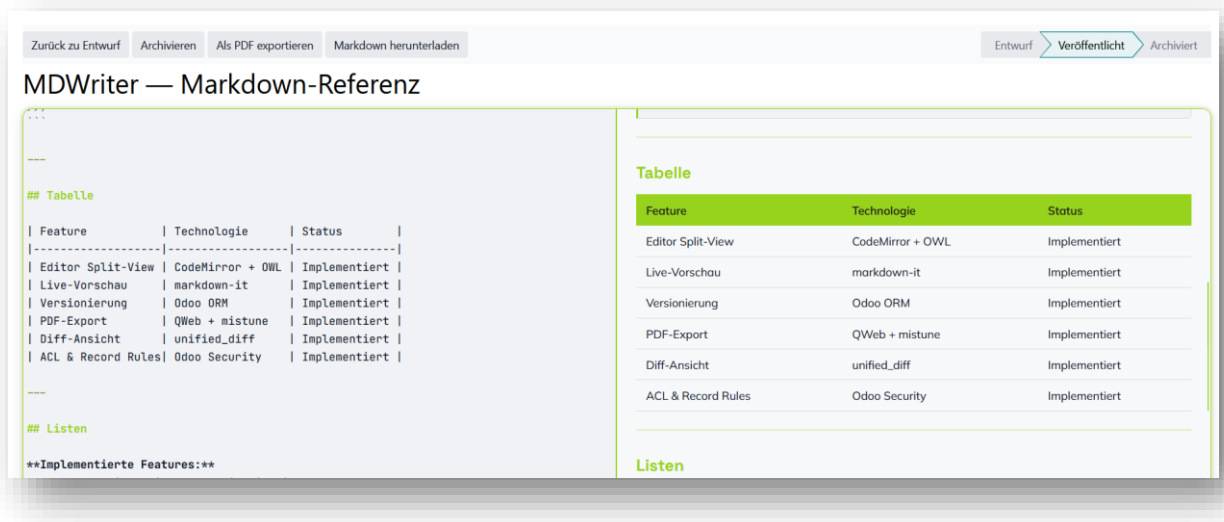
H.3 Dokument erstellen

Über die Schaltfläche „Neu“ wird ein neues Dokument erstellt. Anschließend werden Titel und Markdown-Inhalt eingegeben. Beim Speichern wird automatisch die erste Version des Dokuments angelegt.



H.4 Markdown bearbeiten

Die Bearbeitung erfolgt über den Split-View-Editor. Links wird der Markdown-Text eingegeben, rechts erscheint die Live-Vorschau des gerenderten Dokuments. Unterstützt werden typische Markdown-Elemente wie Überschriften, Listen, Codeblöcke, Links und Tabellen.



H.5 Dokument speichern und versionieren

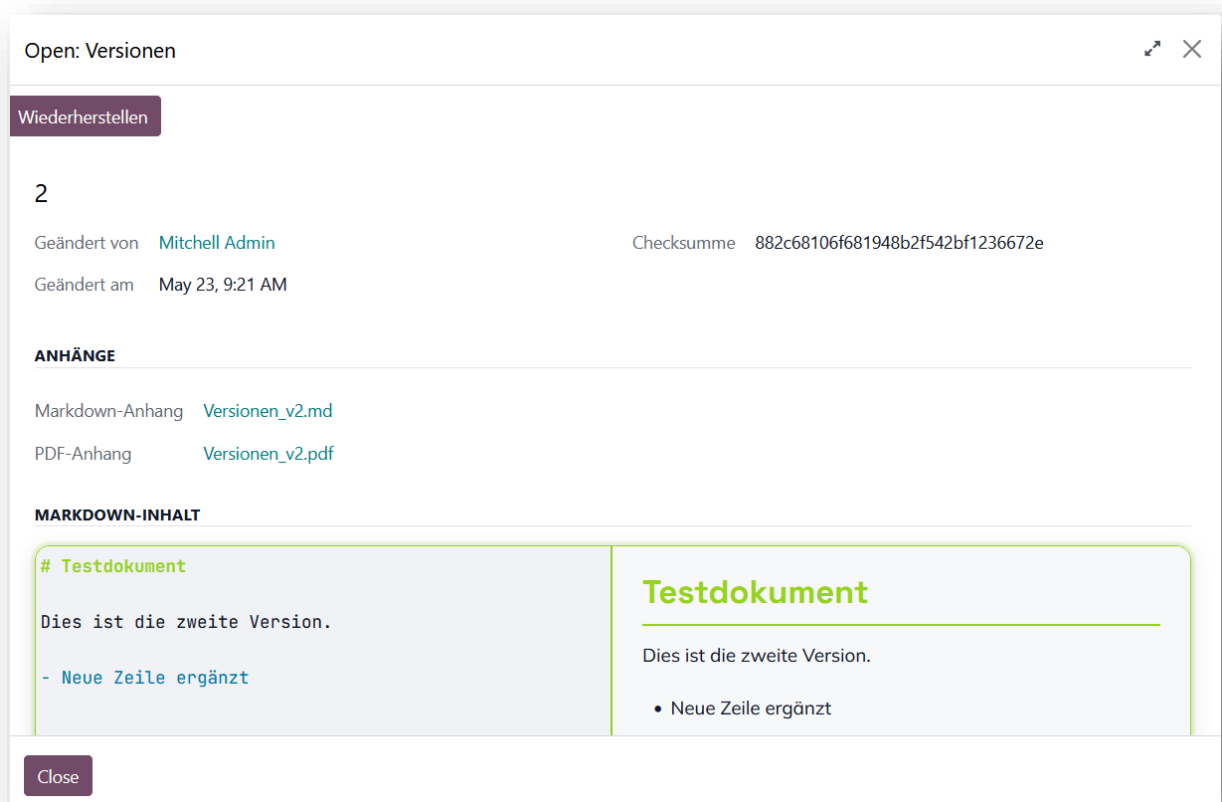
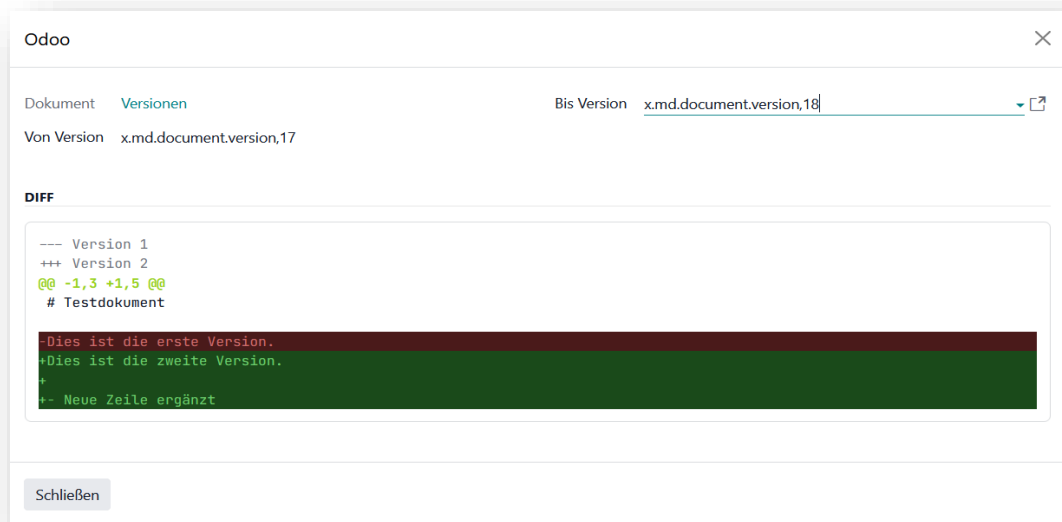
Beim Speichern eines geänderten Markdown-Inhalts legt MDWriter automatisch eine neue Version an. Dadurch bleiben frühere Bearbeitungsstände nachvollziehbar erhalten.

H.6 Dokumentstatus verwalten

Dokumente durchlaufen drei Zustände: Entwurf, Veröffentlicht und Archiviert. Der Status kann über die Schaltflächen im Formulkopf geändert werden. Archivierte Dokumente sind für normale Benutzer nicht mehr bearbeitbar.

H.7 Versionen anzeigen und vergleichen

Über die Versionshistorie können gespeicherte Versionen eines Dokuments eingesehen werden. Der Versionsvergleich zeigt Änderungen zwischen zwei Versionen farblich hervorgehoben an. Über die Schaltfläche „Wiederherstellen“ in der Versionshistorie kann ein früherer Bearbeitungsstand als neue Version des Dokuments übernommen werden.



H.8 PDF exportieren

Über die Exportfunktion kann das aktuelle Dokument als PDF ausgegeben werden. Das PDF enthält den gerenderten Inhalt des Markdown-Dokuments und kann heruntergeladen oder weitergegeben werden.



H.9 Rechte und Zugriff

Normale Benutzer können eigene Dokumente erstellen, bearbeiten und anzeigen. Administratoren besitzen erweiterte Rechte und können alle Dokumente verwalten. Die Rechte werden über das bestehende Odoo-Rechtesystem gesteuert.